Material Flows and On-line Reprocessing in Serpent

Luke Seifert, Alexander Wheeler, Ondrej Chvála
Department of Nuclear Engineering
University of Tennessee at Knoxville, Tennessee, USA
lseifer2@vols.utk.edu, awheel13@vols.utk.edu, ochvala@utk.edu

November 13, 2021

Abstract

The stochastic transport and depletion code Serpent2 allows the capability of feeding and removing materials during depletion. This is a necessary feature for molten salt reactor modeling, but it lacks sufficient documentation and is limited by the static depletion volumes. This paper provides documentation for this on-line reprocessing functionality by presenting the mathematics and constraints behind the various material flow control settings as well as the respective flow rate functions. Multiple solutions for working around the static volume limitation are discussed, specifically through an overflow bucket or repeated restarts of the software with updates of the volumetric data provided.

Keywords: Serpent2, Material flow, Reprocessing, Molten Salt Reactors

Contents

1 Introduction **2**

2 Depletion Volumes **2**

3 Material Flows **3**

4 Example Cases **5**

4.1 Unity Case 6

4.2 Adjusted Case 8

4.3 Multi-flow Case 9

5 Practical MSR Cases **10**

5.1 Single fluid MSR burner 11

5.2 Two fluid MSR breeder 11

6 Conclusions **13**

References **13**

1 Introduction

Serpent2 is a generic Monte-Carlo neutron and photon transport code developed by VTT Research Centre in Finland with depletion and multi-physics coupling capability aimed at nuclear reactor modeling [1], with evolving documentation online [2]. Serpent2 implements material flow features enabling adding or removing core materials during depletion [3]. This is a critical feature for Molten Salt Reactor (MSR) simulations, since any fluid fuelled reactor will produce, at a minimum, off-gas fission product gases. These gases alone have a large reactivity effect due to ^{135}Xe. Material additions and removals, ranging from on-line refueling and separation of a select few fission products up to extensive multi-fluid reprocessing, is of generic interest in liquid fuel reactors. However, these features lack detailed documentation, examples, and test cases.

To supplement this lack of official documentation, this paper seeks to provide an in-depth description of the **rc** function which controls material flows in Serpent, along with working example input decks which demonstrate the functionality. An additional goal is to provide users and developers with a tool for version testing so it will be known what changes are made in the reprocessing functionality of Serpent2. All the relevant files are available on Github [4].

2 Depletion Volumes

Geometric volumes in Serpent remain constant during depletion. When a material is added or removed by a flow, Serpent adjusts the density of the material instead of the volume. For example, imagine an MSR core refueled with a fresh low-enriched uranium bearing salt. The fuel salt density grows and soon becomes nonphysical in the simulation, while in reality the salt volume would grow in an expansion volume of the primary loop.

There are two different methods to correct for constant depletion volumes, though neither are perfect replacements for a dynamic volume adjustment. The first is to use an overflow "bucket" which removes the same volume as what was added. The main drawback of the overflow bucket method is the lack of mixing, since the bucked content is separate from the core and does not mix back with the depleting fuel. The second option is restarting Serpent after a short depletion step

and adjusting the volume and density values, either remixing the overflow materials or ignoring the density rise over a short time step. The main drawback of the second option is the additional code required and the time consumed in code restarts, which in our experience becomes significant due to cross-section loading time. Our previous work [5] used the latter alternative.

Finally, note that the core volume represents only a fraction of the fuel salt depleted in a typical MSR. The salt fraction in a primary circuit out of the core is often larger than within the core. Therefore, the volume of the fuel salt needs to be set manually in the input deck to the actual volume of the fuel salt. This is done either by keyword **vol** on a material card **mat** of the fuel salt or by using a separate card **set mvol** *material zone volume*.

3 Material Flows

Serpent2 implements flows between materials by adding effective decay-like constants into transmutation matrices of materials that are affected by the flows. A brief technical description of this method can be found in [3]. Materials not involved in the flow do not need to actually deplete to use the flow functionality. Serpent does require a presence of fissile material to work, although this fissile material needs not to be depleted nor included in the flows. The reactor power can even be set to 0 without changing the flow functionality.

The mass flows are currently incompatible with the corrector portion of predictor-corrector depletion, requiring keyword **set pcc** to be either 0 or 2 in the input deck. A value of 0 provides constant extrapolation, while 2 uses linear extrapolation, and neither use a corrector. For the cases analyzed, there was no measurable difference between the two different values, so the value of 0 was used in the cases.

A material flow is defined by a keyword **rep**, for reprocessing, followed by *<reprocessing name>*. After this definition come specifications of material flows **mflow** *<mass flow name>* *<nuclide>* *<λ>*. Finally, there is reprocessing control keyword, where the syntax is **rc** *<from material>* *<to material>* *<mass flow name>* *<setting>*. The *<setting>* value changes the equation which governs the flow rate between the materials, and can have a value of 0, 1, or 2.

An additional input which has an effect upon the flow rate, if the *<setting>* value is 1 or 2, is the **dep**, or depletion, information. It is followed by **pro** *<reprocessing name>*, which uses the same reprocessing name as defined in the reprocessing block. The time interval can then be set using different settings such as *daystep* or *daytot*.

Table 1: Material flow and total mass equations for different <*setting*> values

Flow Setting		
0	source mass (g)	$m_{source}(t) = m_{source}(0)$
	destination mass (g)	$m_{destination}(t) = m_{destination}(0) + \lambda m_{source}(0)$
	flow rate (g/s)	$\dot{m}(t) = \lambda m_{source}(0)$
1	source mass (g)	$m_{source}(t) = m_{source}(0)e^{-\lambda t}$
	destination mass (g)	$m_{destination}(t) = m_{destination}(0) + \lambda m_{source}(t)$
	flow rate (g/s)	$\dot{m}(t) = \lambda m_{source}(t)$
2	source mass (g)	$m_{source}(N+1) = m_{source}(N) - \Delta t \lambda m_{source}(N)$
	destination mass (g)	$m_{destination}(N+1) = m_{destination}(N) + \Delta t \lambda m_{source}(N)$
	flow rate (g/s)	$\dot{m}(N) = \lambda m_{source}(N)$

The various equations associated with each <*setting*> value can be seen in Table 1. The mass flow rate $\dot{m}$ [g s^{-1}] depends on source destination masses $m_{source} = V_{source} \times \rho_{source}$ [cm^3], the density of the source from which material is flowing ρ_{source} [g cm^{-3}], the depletion step value N, the time step difference Δt [s] from the first step defined in the **pro** block, and the material flow rate λ [s^{-1}] specified in the **mflow** keyword. Also present in Table 1 are equations for m_{source}, total source material mass, and $m_{destination}$, total destination material mass. Note that these equations will only predict the mass at any given time if there is no depletion.

For each of the <*setting*> values, The mass flow is dependent on the mass present in the source material. In the case where the <*setting*> value is 0, the source material is not depleted. As a result, the flow rate in is constant. This also means that the total mass of the of the entire system will increase over time. On the other hand, both <*setting*> values 1 and 2, deplete the source material and the amount of mass added to the destination is equal to the amount removed from the source.

These equations can also be expressed in terms of original material density ρ [g/cm^3], shown in Table 2, which is indeed the Serpent deck input. This table has one equation for the <*setting*> value of 0 as the mass flow rate is constant over all of the depletion steps. For the other values, 1 and 2, the flow rate changes over the depletion steps at stated before. This change in the flow rate can be seen in the time step iteration counter N in both <*setting*> 1 and 2 equations. It should be

Table 2: Material flow equations for different <*setting*> values

<*setting*>	Material flow equation [g s^{-1}]
0	$\dot{m} \;=\; V_{source}\ \lambda\ \rho^0_{source}$
1	$\dot{m} \;=\; V_{source}\ \rho^0_{source}\ \frac{1-e^{-\lambda\Delta t}}{\Delta t}\ e^{-\lambda N \Delta t}$
2	$\dot{m} \;=\; V_{source}\ \rho^0_{source}\ \lambda\ (1-\lambda\Delta t)^N$

noted that these predictive equations also do not work if there is different material flow streams or material addition via burnup.

If the user causes the source density to go negative with a *setting* of 2, Serpent gives an error stating "Numerical error, try smaller step size." For the *setting* of value of 1, the user cannot cause the source density to be negative. This can be seen in Equation 1 below, which shows the source density with respect to step iteration. Because the values of the initial density, λ, and initial time step must be positive, the density cannot be a negative value.

$$\rho_{source}(N) = \rho^0_{source}(1 - e^{-\lambda\Delta t} + e^{-\lambda\Delta t(N+1)}) \tag{1}$$

4 Example Cases

In order to generate the equations for the flow rates, simple example cases were generated which would allow for analysis of the behaviour of the flows. A sphere of uranium with shells of water and nothing were chosen because the volumes can be calculated by hand easily, they can be created in Serpent easily, and they are close together allowing for better viewing of the geometry to visualize the case.

The main case which was built upon is shown in Figure 1 on the left, where the center is a uranium sphere, the shell directly around it is empty, and the shell on the outer edge is made of water. Two variant cases can be seen in Figure 1 in the middle and on the right, which show how the volume and number of the shells is adjusted to gain further insight into the equations. The exact parameters used can be found in each respective section, though the volume of the fuel remains constant at 15 cm^3 in order to aid in providing visual perspective.

There are also variations in the λ, water density, and time steps. Any variations made are directly noted in each relevant section. From the output of these cases, plots were created of the mass density of the source shell in order to provide visualization of how the different flow settings affect the flow.

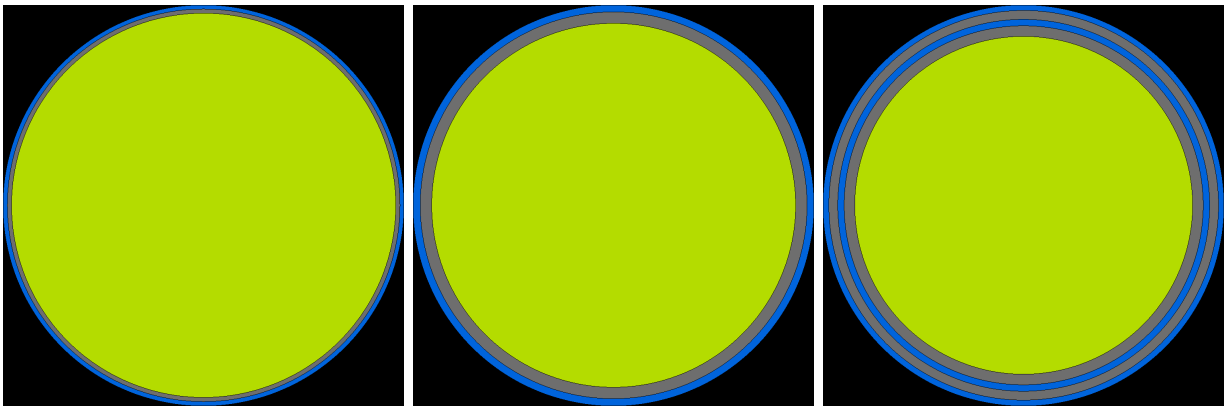

Figure 1: Left: Geometry of a uranium sphere on the inside, an empty shell surrounding it, and a water shell on the outside. Middle: Geometry with variance in volumes. Right: Geometry with four shells of alternating water and empty shells.

In this figure, the volumes for each shell are the same as what they are defined as in Serpent through the **vol** function in material definition. However, the shells could be made to be geometrically different volumes with a defined **vol** value, and the flow would function using the **vol** value and not the actual geometrical volume.

4.1 Unity Case

For the first case, unit volumes were used for the empty shell and the water shell as well as a unit density for water. This allowed parts of the equations to be determined without too many variables to consider. Since this case was the simplest, time variations were also investigated to determine the effect they would have upon the equations. Figure 2 shows the first case evaluated at five second intervals with the flow rate over each time step on the bottom row and the mass density over time on the top row. The different setting values are also shown and labeled in the legend.

The same case with variation in time steps can be seen in Figure 3. Figure 3 is organized the same as Figure 2 except that the time steps used are 0.1, 1, six steps of five, 1, and 0.1 seconds.

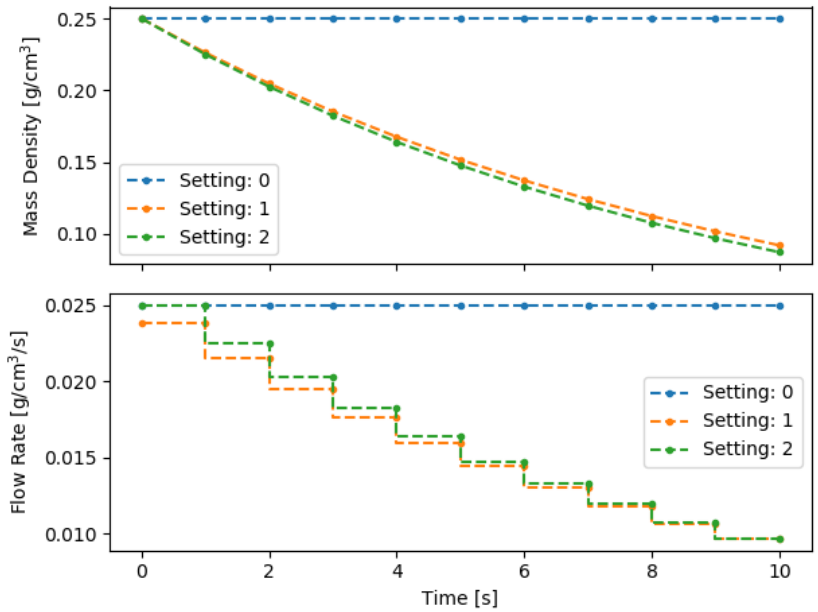

Figure 2: Unity case setup with a constant five second step size. Bottom: Flow rate over time of water to empty shell. Top: Mass density of water shell over time.

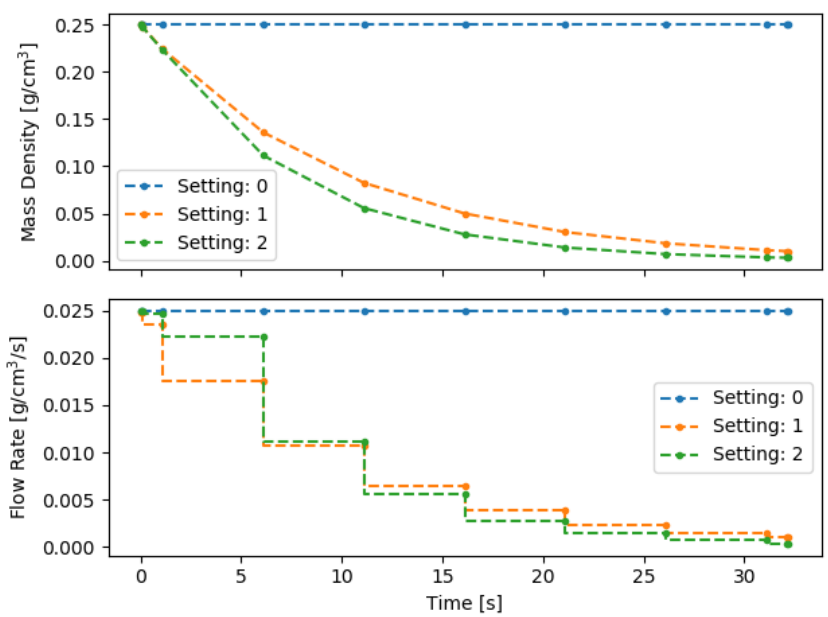

Figure 3: First case setup with a varying time step size of 0.1, 1, six steps of 5, 1, and 0.1. Bottom: Flow rate over time of water to empty shell. Top: Mass density of water shell over time.

4.2 Adjusted Case

The adjusted case uses the unity case with its time step variations and adds adjustments to the volumes of both the shells and the density of the water to non-unity values. This allows for the volume ratio terms and density term to appear in the equations. For this case, the empty shell has a volume of $3\,\mathrm{cm}^3$, the water shell has a volume of $2\,\mathrm{cm}^3$, and the water has a density of $0.25\,\mathrm{g/cm}^3$. Figure 4 shows the empty shell mass density with respect to time on the bottom row as opposed to the source shell mass density.

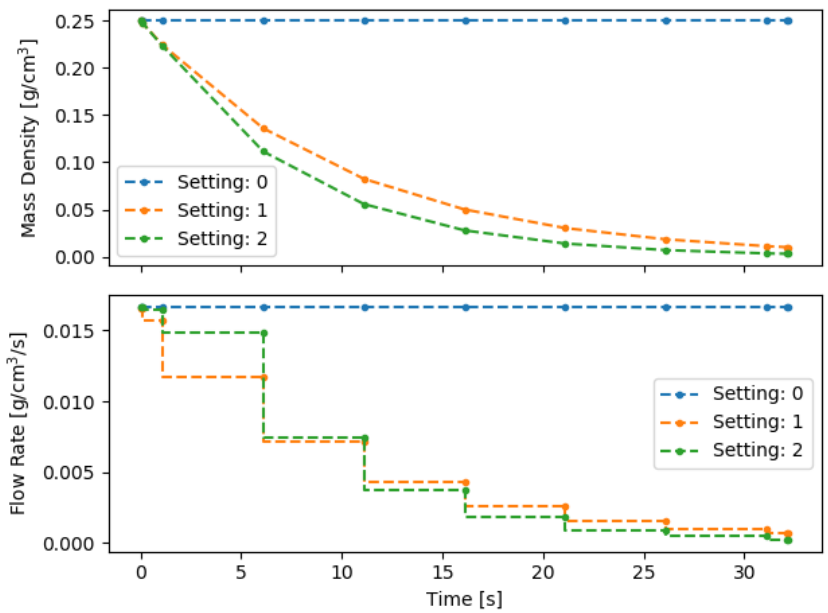

Figure 4: Second case setup with a constant five second step size, variance in volumes, and variance in water density. Bottom: Flow rate over time of water to empty shell. Top: Mass density of water shell over time.

The reprocessing and flow syntax for this case should be the same as the unity case. The only differences between the cases are in the surface and material definitions.

4.3 Multi-flow Case

This case geometry is shown in Figure 1. There are multiple shells with the water moving from the innermost water shell to the innermost empty shell and the outermost water shell to the outermost empty shell. The water densities, shell volumes, and flow rates are different for both flows in the multi-flow case, though they can be made equivalent. For the inner flow, the empty and water shell volumes are $3\,\mathrm{cm}^3$ and $2\,\mathrm{cm}^3$, the water density is $1.7\,\mathrm{g/cm}^3$, and the λ is $0.03\,\mathrm{s}^{-1}$. For the outer flow, the empty and water shell volumes are $4\,\mathrm{cm}^3$ and $5\,\mathrm{cm}^3$, the water density is $0.8\,\mathrm{g/cm}^3$, and the λ is $0.0015\,\mathrm{s}^{-1}$. The flow rates and mass density variance can be seen for the inner water and empty shells in Figure 5, while the flow rates and mass density variance for the outer shells can be seen in Figure 6.

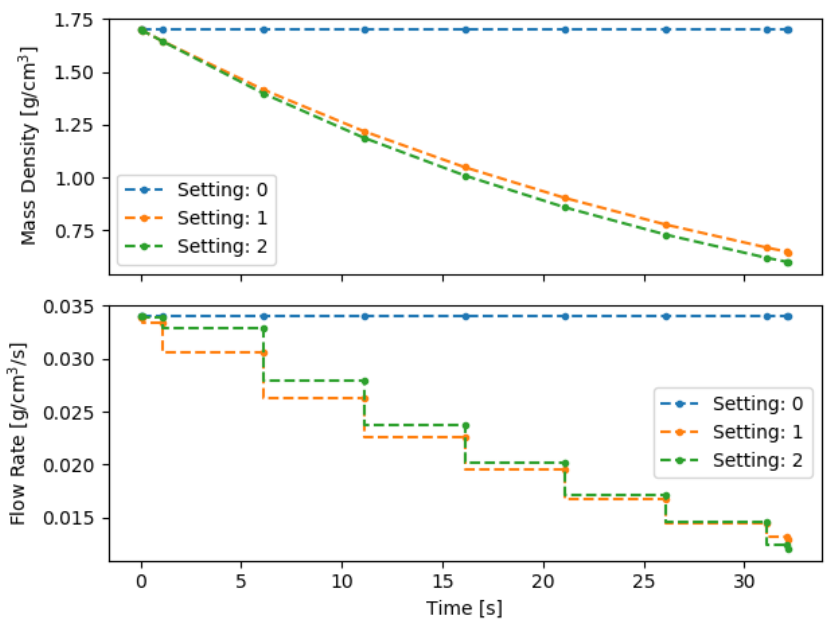

Figure 5: Multi-flow case setup inner shell flow with a constant five second step size, variance in volumes, and variance in water density. Bottom: Flow rate between inner shells over time of water to empty shell. Top: Mass density of inner water shell over time.

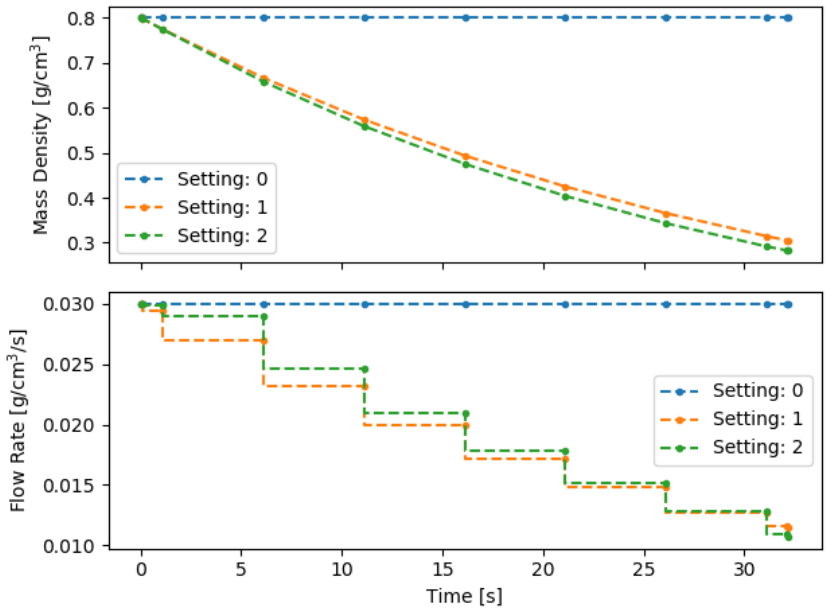

Figure 6: Multi-flow case setup outer shell flow with a constant five second step size, variance in volumes, and variance in water density. Bottom: Flow rate between outer shells over time of water to empty shell. Top: Mass density of outer water shell over time.

Note that if the flow is set to go from an empty shell to a water shell, it will not work since the desired nuclides are not present. For the flow to function properly, some quantity of the nuclides which will flow must be present. A flow system in Serpent will also function if material that flows is added by some other means such as fission, which is done in the practical MSR case. If some other material is added to the empty shell, such as helium, and the **mflow** is set to move only hydrogen and oxygen, it will not function.

In order to maintain the integrity of the depletion matrix in a model with separate flows, such as that used in Figures 5 and 6, incorporating a flow which moves no material but connects the various flow regions will connect the flows in the depletion matrix.

5 Practical MSR Cases

While there are many different MSR concepts proposed, they have significant similarities from the material flow perspective. Any MSR will generate fission gases that will evolve off the core liquid. Most MSRs will add some materials, such as fresh fuel salt, almost continuously. We present two flow setups that pertain to two types of MSRs: a single fluid thermal burner and a two-fluid

thorium breeder. The purpose of these cases is to illustrate how to set the flows, not to discuss any specific design concept. The cases presented here are limited to a single lattice cell.

5.1 Single fluid MSR burner

In the first case, a single fuel channel is used as well as a large volume of low enriched uranium (LEU) refuel salt and a large, nearly empty volume for storing off-gas. The **mflow** function is used to define the flow rates of the refuel salt flow, the off-gas elements that flow, and extra salt to over-flow tank. The flow from the LEU refuel volume to the salt, the flow from the fuel salt to the off-gas volume, and the flow of extra salt to over-flow tank are then defined under the **rep** heading with **rc** subheadings. The refuel salt flow, **mflow U_in**, uses a *<setting>* of 0, while the off-gas flow, **mflow offgasratecore**, uses a *<setting>* of 1. The extra salt flow, **mflow over**, uses *<setting>* of 1 to maintain density of the fuel salt.

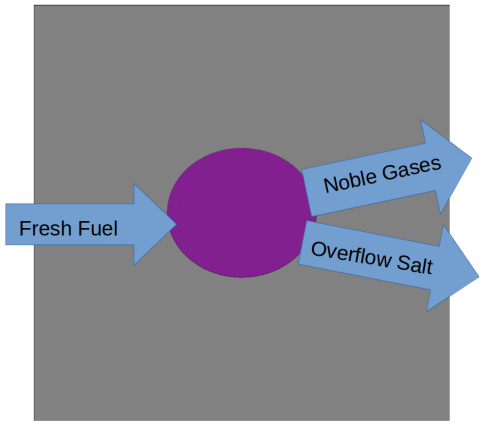

Figure 7: Geometry of a single fluid MSR case, a fuel salt in channel surrounded by a graphite moderator.

Because there are isotopes present that can be flowed to an off-gas at the beginning, by the first step they are present, allowing for the flow to function properly. An alternate version of this was shown in the multi-flow case, but instead of having the flow material develop between steps, it was developed in the same step by a different flow, which did not allow the flow to function properly.

5.2 Two fluid MSR breeder

A two-fluid MSR is exemplified by a thorium molten salt breeder reactor [6]. The core features a hexagonal lattice with a graphite moderator, shown in Figure 8. Cylindrical fuel channels are inside

the graphite hexagon, which is surrounded by a blanket slit. The two fluids are the fuel salt, FLiBe with fissile $^{233}UF_4$, and the blanket salt, FLiBe with fertile $^{232}ThF_4$. Implemented material flows include refueling blanket salt with thorium, removal of bred uranium from the blanket, refueling the fuel salt with the bred uranium, fuel salt off-gassing, and removals of other fission product groups based on their common chemistry. Again, the aim of this example is not to present a specific worked out design, but to illustrate how to set up the material flow topology in a Serpent2 input deck.

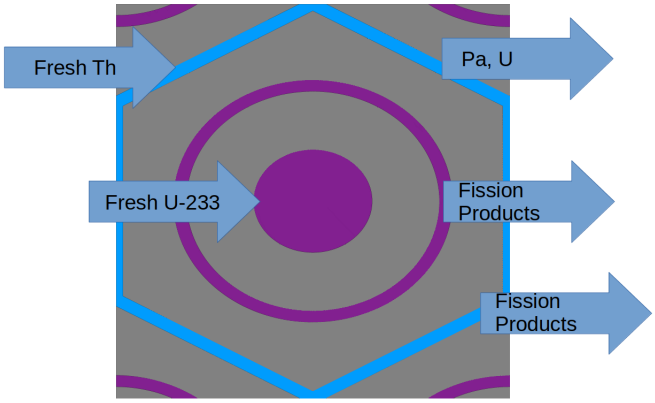

Figure 8: Geometry of a two-fluid MSR case, fuel channels in a hexagonal graphite moderator surrounded by a blanket salt slit.

In this case there are two burnable materials in the geometry, the fuel and blanket salts. Each salt has an **mflows** with *<setting>* of 1 for the various fission products that are processed out. Each fission product group goes to a separate tank based on their respective chemistry, with individual λs that correspond to the respective extraction rates. A pure thorium metal flows into the salt using **mflow thor_in** with *<setting>* of 1. Protactinium and uranium flow from the blanket salt to a uranium stockpile via **mflow U_out** with *<setting>* of 1. Refueling of the core fuel salt is done using **mflow U_in** with a *<setting>* of 2 from the uranium stockpile. This way the uranium stockpile connects the two separate material flows which will collapse the respective depletion matrices into one. This is important in Serpent as there needs to be only one depletion matrix to run.

6 Conclusions

Serpent is a widely used stochastic transport tool for reactor simulations, in particular for MSR peculiarities. However, it has limitations which needs to be understood for correct MSR modelling. Constant volume limits material flow simulation capabilities requiring a users to employ workarounds, and lack of documentation for some of its features means that many users may not be aware of Serpent's capabilities in this area. This paper provides a documentation of how the material flow feature works, example cases of various flow setups in Serpent, and information on limitations of Serpent relevant for MSR modeling. The example cases and scripts for unit tests can be found on GitHub [4].

Acknowledgement

This research was partially funded from US DOE award DE-NE0008793. The authors are grateful for this generous support.

References

[1] J. Leppänen, M. Pusa, T. Viitanen, V. Valtavirta, and T. Kaltiaisenaho, "The Serpent Monte Carlo code: Status, development and applications in 2013," *Annals of Nuclear Energy*, vol. 82, pp. 142–150, 2014.

[2] "Serpent2 online documentation," 2020. [Online]. Available: http://serpent.vtt.fi/mediawiki

[3] M. Aufiero, A. Cammi, C. Fiorina, J. Leppänen, L. Luzzi, and M. Ricotti, "An extended version of the SERPENT-2 code to investigate fuel burn-up and core material evolution of the Molten Salt Fast Reactor," *Journal of Nuclear Materials*, vol. 441, no. 1, pp. 473 – 486, 2013. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0022311513008507

[4] "Github repository for the paper," 2020. [Online]. Available: https://github.com/ondrejch/2020-SerpentFlows

[5] G. Ridley and O. Chvala, "A method for predicting fuel maintenance in once-through MSRs," *Annals of Nuclear Energy*, vol. 110, pp. 265 – 281, 2017. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S030645491730021X

[6] R. C. Robertson, R. B. Briggs, O. L. Smith, and E. S. Bettis, "Two-fluid molten-salt breeder reactor design study, ORNL-4528," Oak Ridge National Laboratory, Tech. Rep., 1970.